

Caso de Estudio: Gestión y Medición de Calidad de Software en una Plataforma de Crédito Digital

Curso: Estándares y Métricas de Calidad en el Desarrollo de Software — Especialización en Desarrollo de Software **Semana 7 — Resultados de aprendizaje:** (1) Establecer las herramientas de calidad de software. (2) Clasificar las técnicas de gestión de la calidad del software.

Nota metodológica: ISO/IEC 25010 es un modelo de características de calidad de **producto**; CMMI nivel 3 es un modelo de áreas de proceso **organizacional**. No son ejes intercambiables. Por eso, en la Sección 4 cada técnica se clasifica con dos columnas independientes: el área de proceso CMMI que la institucionaliza y la característica ISO/IEC 25010 que salvaguarda. Asimismo, los valores numéricos de las Secciones 5 y 6 son ilustrativos para fines pedagógicos; donde se referencia un dato público real (Bancolombia, 2020), se indica explícitamente como tal.

1. Descripción del contexto empresarial y del proyecto de software

Banco Digital Andesur es una entidad financiera de tamaño medio que opera en tres países de la región andina, con un modelo híbrido de banca tradicional y canales digitales. Desde 2022, la organización ejecuta una estrategia de transformación tecnológica orientada a competir con neobancos y fintechs que ofrecen tiempos de aprobación de crédito significativamente menores. Esta presión competitiva llevó al área de Tecnología a reorganizar sus equipos bajo un modelo de squads multidisciplinarios y a priorizar la velocidad de entrega sin sacrificar los controles de seguridad y trazabilidad propios del sector financiero regulado.

El proyecto objeto de este caso es **CrediFlex**, una plataforma de microcréditos digitales que permite a clientes existentes y nuevos solicitar, evaluar y desembolsar créditos de bajo monto desde una aplicación móvil y un portal web, con integración en tiempo real al núcleo bancario, a un motor de scoring crediticio externo y a una pasarela de pagos. El squad responsable está compuesto por doce personas: tres desarrolladores backend (Java/Spring Boot), dos desarrolladores frontend (Angular), dos ingenieros de automatización de pruebas, un ingeniero DevSecOps, un arquitecto de soluciones, un analista de seguridad, un Product Owner y un Scrum Master. El stack tecnológico se construye sobre microservicios Java con Gradle como herramienta de build, exposición de servicios REST y GraphQL, y consumo de un servicio SOAP heredado del núcleo bancario que aún no ha sido modernizado.

Como ejemplo aplicado: durante el sprint 14 del proyecto, el squad debía integrar un nuevo proveedor de scoring crediticio que exponía tanto una API REST como un servicio SOAP de respaldo. Esta heterogeneidad tecnológica —común en bancos con sistemas legados conviviendo con arquitecturas modernas— obligó al equipo a definir desde el inicio una estrategia de pruebas capaz de cubrir múltiples protocolos de integración sin duplicar esfuerzo de automatización, lo cual se convirtió en uno de los criterios determinantes para la selección de herramientas descrita en la Sección 3.

2. Problema identificado relacionado con la calidad del software

Antes de la intervención descrita en este caso, CrediFlex operaba bajo un modelo de aseguramiento de calidad predominantemente manual y reactivo. Cada ciclo de regresión completa tomaba en promedio cinco días hábiles de un analista de pruebas dedicado, lo que obligaba a comprimir las ventanas de despliegue a una cada tres semanas. Los defectos relacionados con reglas de negocio crediticio —cálculo de cupo, tasas y plazos— se detectaban con frecuencia en etapas tardías del ciclo (pruebas de aceptación de usuario o, en el peor de los casos, en producción), generando retrabajo costoso y exposición reputacional ante el regulador financiero.

El problema se agravaba por la ausencia de métricas objetivas de calidad: las decisiones de “listo para producción” dependían del criterio cualitativo del líder técnico, sin umbrales de cobertura, deuda técnica o vulnerabilidades documentados. Adicionalmente, las pruebas de seguridad se ejecutaban de forma puntual antes de auditorías regulatorias, en lugar de integrarse al ciclo de desarrollo, lo que dejaba ventanas de exposición entre despliegues consecutivos. Esta combinación —ciclos de prueba largos, detección tardía de fallas y ausencia de gobierno cuantitativo de calidad— resultaba incompatible con la meta estratégica del banco de igualar la cadencia de entrega de sus competidores fintech.

Como ejemplo aplicado: en el sprint 9, un defecto en el cálculo de la tasa efectiva anual para créditos con plazo superior a 24 meses pasó las pruebas manuales —que solo cubrían los tres escenarios de plazo más frecuentes— y llegó a producción. El error generó 340 solicitudes de crédito con tasas mal calculadas antes de ser detectado por el área de Riesgo, obligando a una corrección manual caso por caso y a un reporte al ente regulador. Este incidente se convirtió en el caso de negocio que justificó la inversión en automatización descrita en este documento.

3. Herramientas de calidad seleccionadas y justificación de su elección

La selección de herramientas siguió tres criterios: alineación con el stack tecnológico Java/Gradle ya adoptado, capacidad de generar documentación viva trazable a requisitos de negocio (exigencia regulatoria) y soporte multiprotocolo para convivir con servicios SOAP heredados junto a REST y GraphQL modernos. Para la capa de pruebas de aceptación end-to-end se eligió **SerenityBDD** sobre Gherkin y Cucumber, dado que su adopción del patrón Screenplay facilita pruebas mantenibles a medida que crece el número de escenarios, y su reportería de documentación viva sustituye los antiguos documentos de evidencias manuales, cumpliendo simultáneamente el objetivo de trazabilidad regulatoria y el de reducción de esfuerzo documental.

Para la capa de integración se seleccionó **Karate Framework**, ejecutado sobre el mismo stack Java/Gradle del proyecto, por su capacidad nativa de probar servicios REST, GraphQL y SOAP con una sintaxis BDD unificada, eliminando la necesidad de mantener herramientas separadas como SoapUI para el servicio heredado del núcleo bancario. Se complementó con **TestContainers** para pruebas de componentes que

requieren una base de datos real en lugar de dobles de prueba, y con **Spring Cloud Contract** para pruebas de contrato entre CrediFlex y el servicio de scoring externo, anticipando incompatibilidades antes de que el proveedor despliegue cambios en su API. En la capa unitaria se mantuvo **JUnit 5** con **Mockito**, estándar de facto del ecosistema Java, integrado con **SonarQube** para análisis estático y aplicación de quality gates automatizados sobre cobertura y deuda técnica. Finalmente, para pruebas especializadas se incorporaron **JMeter** para pruebas de rendimiento y **OWASP ZAP** para análisis dinámico de seguridad (DAST) dentro del pipeline de CI/CD, dado que ambas son herramientas de código abierto con amplia adopción en banca digital y permiten ejecución automatizada en cada despliegue a ambientes no productivos.

Como ejemplo aplicado: la integración del proveedor de scoring crediticio mencionada en la Sección 1 se probó completamente con Karate Framework, escribiendo un único conjunto de escenarios .feature que ejecutaba tanto contra el endpoint REST principal como contra el servicio SOAP de respaldo, reduciendo de dos suites de automatización separadas (la alternativa con SoapUI más RestAssured) a una sola, con una disminución estimada del 40% en horas-ingeniero de mantenimiento de pruebas para ese componente.

4. Técnicas de gestión de calidad aplicadas, clasificadas según ISO/IEC 25010 y CMMI nivel 3

Las técnicas de gestión de calidad institucionalizadas en CrediFlex operan en dos planos complementarios: el plano de **proceso**, gobernado por las áreas de proceso de CMMI nivel 3 (madurez “Definido”, donde las prácticas dejan de depender de la disciplina individual y se estandarizan a nivel organizacional), y el plano de **producto**, gobernado por las características de calidad de ISO/IEC 25010. Una misma técnica puede institucionalizar un proceso CMMI mientras protege simultáneamente más de una característica ISO 25010; la siguiente tabla documenta esa relación sin forzar una correspondencia uno a uno.

Técnica de gestión de calidad	Área de proceso CMMI nivel 3	Característica(s) ISO/IEC 25010
TDD (Test-Driven Development)	Solución Técnica (TS)	Confiabilidad (tolerancia a fallos), Mantenibilidad (testabilidad)
BDD/ATDD con Gherkin + SerenityBDD	Desarrollo de Requisitos (RD), Validación (VAL)	Adecuación funcional (corrección), Usabilidad (comprensibilidad vía documentación viva)
Pruebas de contrato (Spring Cloud Contract)	Integración de Producto (PI)	Compatibilidad (interoperabilidad)
Pruebas de componentes (TestContainers)	Verificación (VER)	Confiabilidad, Adecuación funcional

Técnica de gestión de calidad	Área de proceso CMMI nivel 3	Característica(s) ISO/IEC 25010
Análisis estático y quality gates (SonarQube)	Aseguramiento de Calidad de Proceso y Producto (PPQA)	Mantenibilidad (analizabilidad, modularidad), Seguridad
Pruebas de seguridad (OWASP ZAP, Hacking Continuo)	Gestión de Riesgos (RSKM)	Seguridad (confidencialidad, integridad, autenticidad)
Pruebas de rendimiento (JMeter)	Verificación (VER)	Eficiencia de desempeño
Automatización CI/CD y configuración del pipeline	Gestión de Configuración (CM), Definición de Proceso Organizacional (OPD)	Portabilidad (capacidad de instalación), Mantenibilidad (modularidad)

Esta doble clasificación tiene valor práctico más allá del ejercicio académico: cuando un equipo discute si una técnica “vale la pena”, la columna CMMI responde si la organización gana madurez de proceso reproducible, mientras la columna ISO 25010 responde qué atributo concreto experimentará el usuario final. En CrediFlex, por ejemplo, la justificación de negocio para invertir en pruebas de contrato no fue “madurar Integración de Producto” en abstracto, sino evitar incidentes de compatibilidad con el proveedor externo de scoring, lo cual es la consecuencia tangible de esa área de proceso.

Como ejemplo aplicado: cuando el equipo de arquitectura propuso adoptar TDD de forma obligatoria para los componentes de cálculo financiero (tasas, cupos, amortización), la decisión se evaluó en ambos planos: a nivel CMMI, la práctica institucionaliza Solución Técnica al exigir que el diseño del componente nazca de un caso de prueba, reduciendo la dependencia de un único desarrollador senior; a nivel ISO 25010, el resultado esperado era mejorar la Confiabilidad del cálculo financiero, justamente el atributo cuya falla había generado el incidente del sprint 9.

5. Métricas utilizadas con valores numéricos de ejemplo

Las métricas siguientes corresponden a valores ilustrativos definidos para este caso de estudio, comparando el estado “antes” (proceso predominantemente manual, sprint 1-8) con el estado “después” (proceso automatizado con la pila descrita en la Sección 3, sprint 15-20). El umbral de cobertura $\geq 70\%$ y de deuda técnica menor a 2 días reflejan, como referencia de industria, las reglas de quality gate que Bancolombia declara aplicar públicamente en su práctica de pruebas continuas (Puerta Ospina, 2020); los demás valores son contruidos para este caso ficticio.

Métrica	Antes (manual)	Después (automatizado)
Densidad de defectos (defectos / KLOC)	8,4	2,1
Cobertura de pruebas unitarias	35%	76%
Cobertura de pruebas de integración	0%	62%
Deuda técnica acumulada (días-persona)	14	1,8
Duración del ciclo de regresión completa	5 días	45 minutos
Defectos críticos detectados antes de producción	60%	96%
Vulnerabilidades críticas abiertas al desplegar	9	0
Frecuencia de despliegue a producción	1 cada 3 semanas	4 por semana
Tiempo medio de resolución (MTTR) de incidentes	18 horas	3 horas

Como ejemplo aplicado: la métrica de densidad de defectos se calculó dividiendo el número de defectos reportados en producción durante un trimestre entre las líneas de código efectivas del módulo de cálculo crediticio (medidas en miles de líneas, KLOC). El descenso de 8,4 a 2,1 defectos/KLOC no se atribuyó únicamente a la automatización de pruebas, sino al efecto combinado con TDD: al exigir que cada función de cálculo tuviera su prueba unitaria antes de la implementación, el equipo detectó proactivamente casos límite (plazos atípicos, montos en el extremo superior del rango permitido) que antes solo se descubrían mediante reportes de usuarios reales.

6. Resultados obtenidos: detección temprana de fallas, reducción de costos y tiempos

La adopción combinada de las herramientas y técnicas descritas produjo tres categorías de resultados medibles. En primer lugar, la **detección temprana de fallas** mejoró sustancialmente: el porcentaje de defectos críticos identificados antes de llegar a producción pasó del 60% al 96%, lo que representa una reducción del 90% en la proporción de defectos críticos que escapan al ambiente productivo (de 40% de fuga a 4%). Este desplazamiento del punto de detección hacia etapas tempranas del ciclo —principio conocido como *shift-left testing*— se explica principalmente por la combinación de TDD a nivel unitario y BDD a nivel de aceptación, que en conjunto cubren tanto la lógica interna de los componentes como el comportamiento observable desde la perspectiva del negocio.

En segundo lugar, la **reducción de costos y tiempos** se evidenció en el ciclo de regresión: el tiempo de ejecución pasó de 5 días-persona a 45 minutos de ejecución automatizada sin intervención manual, una reducción superior al 99% en tiempo de ejecución y aproximadamente 85% en costo total considerando el esfuerzo de mantenimiento de la suite automatizada frente al costo del analista dedicado a tiempo completo que antes ejecutaba las pruebas manuales. La frecuencia de despliegue se multiplicó por doce (de uno cada tres semanas a cuatro por semana), habilitando una cadencia de entrega de valor más competitiva frente a actores fintech del mercado. Finalmente, el tiempo medio de resolución de incidentes en producción (MTTR) se redujo de 18 a 3 horas, atribuible en parte a la trazabilidad que aporta la documentación viva de SerenityBDD, que permite a los equipos de soporte identificar rápidamente qué criterio de aceptación quedó invalidado por un cambio reciente.

Como ejemplo aplicado: en el sprint 20, un cambio en la lógica de aprobación automática para créditos preaprobados fue detectado como defectuoso por la suite de pruebas de contrato (Spring Cloud Contract) antes de fusionarse a la rama principal, evitando que una incompatibilidad con el proveedor de scoring se propagara a producción. El costo estimado de no haber detectado este defecto —basado en el incidente comparable del sprint 9 que generó 340 casos de remediación manual— se calculó en aproximadamente 120 horas-persona de retrabajo evitadas, frente a las 6 horas-persona que tomó ajustar el contrato antes del despliegue.

7. Conclusiones sobre el impacto en la calidad del producto y el posicionamiento competitivo de la empresa

El caso de CrediFlex ilustra que la calidad de software en un contexto financiero regulado no puede gestionarse exclusivamente desde la perspectiva de cumplimiento normativo ni exclusivamente desde la velocidad de entrega: ambas dimensiones convergen cuando las técnicas de gestión de calidad se institucionalizan como proceso (CMMI) y se orientan hacia atributos verificables de producto (ISO/IEC 25010). La mejora simultánea en densidad de defectos, cobertura y frecuencia de despliegue muestra que la dicotomía tradicional entre “probar más” y “entregar más rápido” se resuelve mediante automatización bien gobernada, no mediante la reducción del alcance de las pruebas.

Desde la perspectiva de posicionamiento competitivo, la capacidad de Banco Digital Andesur de pasar de un despliegue cada tres semanas a cuatro despliegues semanales sin incrementar el riesgo operativo —medido por la reducción de vulnerabilidades críticas abiertas y de defectos en producción— constituye una ventaja estratégica directamente comparable con la cadencia de entrega de competidores fintech nativos digitales. La trazabilidad regulatoria, lograda mediante la documentación viva de las pruebas de aceptación, resuelve además una tensión específica del sector financiero: la necesidad de auditar el comportamiento del software sin que ello implique volver a procesos manuales que ralentizan la entrega.

Finalmente, este caso evidencia un principio transferible a otros contextos: la madurez de proceso (CMMI) sin métricas de producto (ISO 25010) corre el riesgo de optimizar procedimientos que no impactan al usuario final, mientras que perseguir

características de producto sin institucionalización de proceso (por ejemplo, dependiendo de la disciplina individual de un desarrollador para aplicar TDD) no es sostenible a escala organizacional. La sostenibilidad del programa de calidad de CrediFlex dependerá, hacia adelante, de mantener ambos planos articulados a medida que el banco extienda esta estrategia a otras plataformas de su portafolio digital.

Referencias

Puerta Ospina, M. (2020). *Pruebas Continuas de Software: Una cultura estratégica para responder a la velocidad que requiere un banco con calidad y seguridad*. Bancolombia Tech. Medium.